

Sistema de Gestão Odontológica

Trabalho Incremental de Software 1 (TIS1) • Seminário de Projeto Final

Equipe Desenvolvedora (Matrícula • Nome):

202203500 • Amanda Almeida dos Santos
202203506 • Fabrício Silva Dias
202203508 • Filipe Augusto Lima Silva
202203509 • Guilherme Luis Andrade Borges
202203512 • Isabela de Queiroz Rodrigues

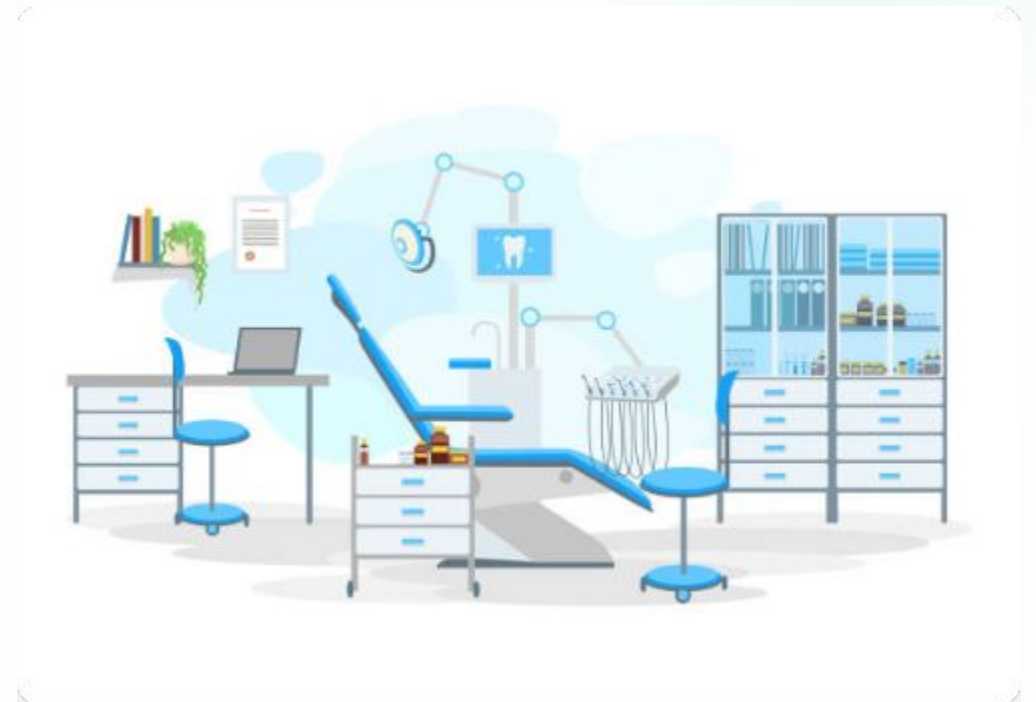
Universidade Federal de Goiás (UFG)

Instituto de Informática • Curso de Desenvolvimento Full Stack
Orientador: Prof. Dirson Santos de Campos
E-mail de Contato: dirson_campos@ufg.br

Definição da Microaplicação

O sistema foi planejado e projetado para controlar integralmente os fluxos de atendimento diário, automatizando os pontos mais críticos da clínica:

-  **Gestão de Consultas:** Ciclo completo de agendamento, atualização de status e histórico de sessões.
-  **Histórico de Atendimentos:** Registro detalhado e acumulativo de procedimentos vinculados a cada consulta executada.
-  **Modelagem Limpa:** Estrutura robusta composta por 4 tabelas principais inter-relacionadas (Paciente, Dentista, Consulta e Procedimento).



Visão Arquitetural do Sistema

Arquitetura Monolítica Modular

- **Conceito do Monolito Modular:** Implementação lógica em Spring Boot que segmenta domínios independentes de forma modularizada no backend.
- **Baixo Acoplamento:** O código é compilado de forma centralizada, mantendo os domínios (Paciente, Consulta, Dentista) blindados e independentes.
- **Manutenibilidade Facilitada:** Testes de integração isolados e facilidade de refatoração comparados a monolitos clássicos.

Comunicação Desacoplada (SPA)

- **Interface Reativa Angular:** SPA (Single Page Application) em TypeScript que renderiza e processa fluxos no lado do cliente.
- **Consumo RESTful:** Interação exclusiva por rotas HTTP padronizadas transferindo estruturas JSON.
- **Documentação Swagger:** Mapeamento dinâmico e interativo dos endpoints REST que serve de roteiro exato para a integração do frontend.

Requisitos Funcionais (Parte 1)

Código	Entidade / Funcionalidade	Descrição Técnica	Prioridade
RF01	CRUD de Pacientes	Gerenciamento completo de Pacientes (ID, Nome, CPF único, Data de Nascimento, Telefone, Endereço).	ALTA
RF02	CRUD de Dentistas	Gerenciamento completo de Dentistas (ID, Nome, CRO único, Especialidade, Telefone, E-mail).	ALTA
RF03	Agendamento de Consulta	Registrar Consultas associando um Paciente, um Dentista, Data, Hora e Status (Agendada, Concluída, Cancelada).	ALTA

Requisitos Funcionais (Parte 2)

Código	Entidade / Funcionalidade	Descrição Técnica	Prioridade
RF04	Impedir Choque de Horário	O sistema deve validar e bloquear o agendamento de mais de uma consulta para o mesmo dentista no mesmo horário.	ALTA
RF05	Registro de Procedimento	Vincular múltiplos procedimentos odontológicos em uma consulta (Descrição, Valor financeiro, observações clínicas).	MÉDIA
RF06	Histórico Clínico	Permitir a visualização rápida e cronológica do histórico de atendimentos completo de um paciente.	MÉDIA

| Requisitos Não Funcionais (RNF)



RNF01 • Backend Spring Boot

Core de negócios desenvolvido obrigatoriamente em Java 17+, com arquitetura do framework Spring Boot 3+.



RNF02 • Frontend Angular

Componentização e interface SPA modularizadas em TypeScript sob o ecossistema reativo do Angular.



RNF03 • Banco de Dados Relacional

Armazenamento transacional gerenciado pelo PostgreSQL, com mapeamento de tabelas via Hibernate e Spring Data JPA.



RNF04 • Modelo Arquitetural API

Rotas RESTful limpas para tráfego e serialização de dados, documentadas e expostas dinamicamente via Swagger UI.

| Documentação e Artefatos do Sistema

Modelagem Estática

- **Diagrama Entidade-Relacionamento (DER):** Modelagem das tabelas do banco de dados relacional e definição de chaves primárias e
- **Diagrama de Classes UML:** Mapeamento da estrutura lógica orientada a objetos das entidades de domínio e as camadas de serviço/repositório.
- **Descrição Textual de Requisitos:** Especificação detalhada de cada comportamento do sistema e suas restrições de tecnologia.

Modelagem Dinâmica e UX

- **Diagrama de Casos de Uso:** Representação das interações dos atores (Recepcionistas, Administradores, Dentistas) com a aplicação.
- **Diagrama de Sequência:** Ilustração detalhada do fluxo dinâmico temporal e das mensagens trocadas na criação de um agendamento.
- **Protótipos Figma:** Telas projetadas em alta fidelidade com as diretrizes visuais e reativas que guiarão os componentes do Angular.

| Diagrama de Casos de Uso (UML)



Recepcionista / Admin

Atua como ator principal do agendamento técnico. Suas responsabilidades diretas incluem as operações CRUD de **Gerenciar Pacientes**, **Gerenciar Dentistas** e o fluxo de **Agendar Consultas**.



Caso de Uso Compartilhado

O caso de uso **Agendar Consulta** e a ação de **Consultar Histórico do Paciente** são fluxos transversais que integram os dados inseridos pela recepção com a visualização técnica do corpo médico.



Dentista

Ator clínico focado no atendimento direto. Possui permissões específicas de visualização e alteração para **Consultar Histórico Clínico** e **Registrar Procedimento** executados no paciente.

Diagrama Entidade-Relacionamento

Entidades Fortes (Cadastros)

- **PACIENTE:** Identificado por `id` (PK). Possui os campos de cadastro básico como `nome` (String), `cpf` (String - Único), `dataNascimento` (Date) e `telefone` (String).
- **DENTISTA:** Identificado por `id` (PK). Contém atributos exclusivos de atuação clínica como `nome` (String), `cro` (String - Registro Profissional Único) e `especialidade` (String).
- **Independência Física:** Ambos os cadastros funcionam isoladamente nas tabelas, sem carregar dependências externas estruturais.

Entidades Fracas e Relacionamentos

- **CONSULTA (Relacionamento 1..*):** Estrutura central vinculada a um Paciente (FK `paciente\_id`) e a um Dentista (FK `dentista\_id`). Registra `dataHora` (DateTime) e `status` (String).
- **PROCEDIMENTO:** Depende de uma consulta existente (FK `consulta\_id`). Armazena o descritivo de serviços como `id` (PK), `descricao` (String) e `valor` (Decimal).
- **Fluxo de Integridade:** Chaves estrangeiras estruturadas para cascatas lógicas que garantem a coesão referencial no PostgreSQL.

Estrutura de Classes e Camadas

Modelagem de Domínio (Entidades)

- **Paciente & Dentista:** Mapeados diretamente como `@Entity` pelo Hibernate, respeitando os tipos Java como `LocalDate` para datas.
- **Consulta:** Agrega as entidades de domínio `Paciente` e `Dentista`, possuindo relacionamento `@ManyToOne` e um Enum `@Enumerated` para `StatusConsulta`.
- **Procedimento:** Mapeado com dependência direta de `@ManyToOne` para `Consulta`, utilizando `BigDecimal` para controle monetário.

Camada de Serviços e Controle

- **ConsultaService:** Camada de lógica de negócio e validações técnicas. Métodos fundamentais: `agendarConsulta(dados)`, `cancelarConsulta(id)` e `listarPorPaciente(id)`.
- **Interfaces Repositories:** Declaração de herança do `JpaRepository` do Spring Data, disponibilizando persistências nativas sem necessidade de query manual.
- **Injeção de Dependências:** Acoplamento gerenciado pelo Spring Core através de declarações de inversão de controle.

Processo de Agendamento e Validação



| Interfaces de Usuário (Angular & Figma)



Dashboard Inicial

Abertura com indicadores consolidados das atividades do dia. Mostra o quantitativo de consultas ativas, total de profissionais de plantão e atalhos estruturados para novo agendamento rápido.



Tabela de Pacientes

Lista inteligente paginada com buscas reativas no Angular por nome ou CPF. Possui botão flutuante para abrir o modal de cadastro rápido mapeado com formulários reativos do framework.



Agenda e Calendário

Visualização em formato de calendário semanal dinâmico. Permite o clique intuitivo do usuário sobre qualquer slot ou célula de horário disponível para disparar a tela de marcação de nova consulta.



Ficha Prontuário

Ambiente privativo para atuação do dentista durante o atendimento. Viabiliza a visualização do histórico progressivo do paciente e o registro pontual de novos procedimentos na sessão atual.

Estratégia Híbrida de Testes



Testes Unitários no Service

Implementados com JUnit 5 e Mockito. Focam no isolamento e simulação das regras de negócio complexas, como o bloqueio contra agendamentos simultâneos (RF04).



Testes de Integração de API

Executados através de MockMvc no Spring Boot. Simula chamadas JSON reais para validar o mapeamento de controllers REST e a fidelidade dos HTTP Status Codes.



Testes Manuais E2E

Validação direta no navegador integrada à API. Foca na experiência de ponta a ponta, avaliando validações reativas de formulários Angular (como CPF inválido).



Critérios de Aceite

A aprovação técnica do software está estritamente vinculada ao sucesso da persistência íntegra do banco PostgreSQL e ausência de travamentos de interface.

| Referências Bibliográficas

-  *Trabalho Incremental de Software 1 (TIS1) — Desenvolvimento Full Stack*. Universidade Federal de Goiás, Instituto de Informática. Goiânia, 2026.
-  *Documentação de Software: Sistema de Gerenciamento Odontológico*. Instituto de Informática. Disciplina: Trabalho Incremental de Software 1 (TIS1). Goiânia, 2026.
-  *Slides Seminário Modelo — Desenvolvimento Full Stack*. Universidade Federal de Goiás, Instituto de Informática. Goiânia, 2026.